

Ellenfél alapú támadások (FGSM, PGD)
implementációja
konvolúciós neurális hálózatokhoz
interaktív tesztkörnyezetben

Gabrity Gábor
Neptun kód: D9V09Z

2026. május 25.

Kivonat

A mesterséges intelligencia rohamos fejlődésével párhuzamosan kritikus kérdéssé vált a modellek biztonsága és megbízhatósága. Kutatások kimutatták, hogy a modern konvolúciós neurális hálózati architektúrák sebezhetőek az ellenfél alapú támadásokkal szemben, mint például az FGSM (Fast Gradient Sign Method) és a PGD (Projected Gradient Descent). Ezek az algoritmusok a bemeneti adatot képesek olyan módon – az emberi szem számára szinte észrevétlenül – módosítani, hogy a nagy megbízhatóságú modelleket is megtévesztik.

Szakdolgozatom célja egy olyan interaktív tesztkörnyezet fejlesztése, amely grafikus felhasználói felületen keresztül teszi lehetővé ezen biztonsági rések vizuális elemzését. A projekt fókuszában az AI Red Teaming módszertan áll: a fejlesztő vagy biztonsági szakember a szoftver segítségével szisztematikusan „támadja” a választott képfelismerő modellt, hogy feltárja annak gyenge pontjait.

A keretrendszer több elismert támadási algoritmust implementál, köztük a gyors, egylépéses FGSM-et és a robusztusabb, iteratív PGD-t. A felhasználó számára lehetőség nyílik mind általános tévesztés (non-targeted), mind specifikus célosztályra irányuló (targeted) támadások végrehajtására, miközben interaktív módon, csúszkák segítségével szabályozhatja a perturbáció mértékét és az iterációk számát.

Az alkalmazás hármas vizuális megjelenítést alkalmaz: megjeleníti az eredeti képet, a generált zajt (perturbációt) és a végső, támadott képet, miközben mutatja a modell predikcióinak változását. A rendszer kiegészülhet egy modellvédelmi modulal is, amely különböző képfeldolgozási szűrők – például homályosítás – segítségével próbálja megsemmisíteni a beágyazott zajt, mérve ezzel a modell védekezőképességét.

Tartalomjegyzék

1. Ellenfél alapú támadások	3
1.1. FGSM (Fast Gradient Sign Method)	3
1.1.1. Matematikai háttér	3
1.1.2. Az algoritmus lépései	3
1.1.3. Célzott változat	3
1.1.4. Előnyök és korlátok	4
1.2. PGD (Projected Gradient Descent)	4
1.2.1. Matematikai háttér	4
1.2.2. Az algoritmus lépései	4
1.2.3. Célzott változat	4
1.2.4. Hiperparaméterek hatása	5
1.2.5. Előnyök és korlátok	5

1. Ellenfél alapú támadások

A mély neurális hálózatok – különösen a képfelismerésben alkalmazott konvolúciós architektúrák – rendkívül érzékenyek a bemeneti adatok apró, célzott módosításaira. Az ilyen módosításokat *ellenfél alapú perturbációknak* (adversarial perturbations) nevezzük, az ezeket előállító eljárásokat pedig *ellenfél alapú támadásoknak* (adversarial attacks) [1].

A támadások alapelve a következő: adott egy f osztályozó modell, egy x eredeti bemenet és annak helyes y címkéje. A cél egy olyan x_{adv} módosított bemenet előállítása, amelyre $f(x_{\text{adv}}) \neq y$, miközben a perturbáció $\delta = x_{\text{adv}} - x$ valamilyen norma szerint kicsi marad: $\|\delta\|_p \leq \epsilon$. A gyakorlatban leggyakrabban az ℓ_∞ normát alkalmazzák, amely a pixelenkénti maximális eltérést korlátozza.

A következő alfejezetekben a két legelterjedtebb gradiens alapú támadási módszert mutatom be részletesen.

1.1. FGSM (Fast Gradient Sign Method)

Az FGSM algoritmust Goodfellow és társai javasolták 2015-ben [1]. Az eljárás egyetlen lépésben állítja elő a perturbációt a veszteségfüggvény bemenetre vonatkozó gradiensének előjelét felhasználva.

1.1.1. Matematikai háttér

Legyen $J(\theta, x, y)$ a modell veszteségfüggvénye (például keresztentropia), ahol θ a hálózat tanult paraméterei, x a bemeneti kép és y az eredeti címke. Az FGSM a veszteségfüggvény x -re vonatkozó gradiensét számítja ki visszaterjesztéssel (backpropagation), majd a perturbációt a gradiens előjelének ϵ -szorosával képezi:

$$x_{\text{adv}} = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$$

A $\text{sign}(\cdot)$ függvény elemenként az előjelet adja vissza (-1 , 0 vagy $+1$), így a perturbáció minden pixel esetén pontosan $\pm\epsilon$ lesz. Ez biztosítja, hogy a módosítás az ℓ_∞ norma szerint pontosan ϵ nagyságú.

1.1.2. Az algoritmus lépései

1. Előreterjesztés (forward pass): a modell kiszámítja az x bemenethez tartozó predikciót és a veszteségfüggvény $J(\theta, x, y)$ értékét.
2. Visszaterjesztés (backward pass): kiszámítjuk a gradienst $\nabla_x J(\theta, x, y)$.
3. Perturbáció előállítása: $\delta = \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$.
4. Támadott kép: $x_{\text{adv}} = \text{clip}(x + \delta, 0, 1)$, ahol a clip függvény biztosítja, hogy a pixelértékek a $[0, 1]$ tartományban maradjanak.

1.1.3. Célzott változat

Az alapváltozat (*non-targeted*) a helyes osztály veszteségét maximalizálja. A *targeted* változatban egy y_{target} célosztályt adunk meg, és a perturbációt úgy állítjuk elő, hogy a modell ebbe az osztályba sorolja a képet:

$$x_{\text{adv}} = x - \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y_{\text{target}}))$$

Az előjel megfordul, mert ebben az esetben a célosztályhoz tartozó veszteséget *minimalizáljuk*.

1.1.4. Előnyök és korlátok

Az FGSM fő előnye a sebessége: egyetlen gradiens-számítás elegendő. Hátránya, hogy egyetlen nagy lépést tesz, így kisebb ϵ értékek mellett gyakran kevésbé hatékony, mint az iteratív módszerek. Emellett a perturbáció iránya nem feltétlenül optimális, csupán a gradiens előjelét követi.

1.2. PGD (Projected Gradient Descent)

A PGD algoritmust Madry és társai vezették be 2018-ban [2] az FGSM iteratív általánosításaként. A módszert a szakirodalom gyakran a „legerősebb elsőrendű támadásnak” tekinti az ℓ_∞ korlátos perturbációk között.

1.2.1. Matematikai háttér

A PGD a következő iteratív frissítési szabályt alkalmazza, $t = 0, 1, \dots, T - 1$ lépésben:

$$x^{(t+1)} = \Pi_{\mathcal{B}_\epsilon(x)} \left(x^{(t)} + \alpha \cdot \text{sign}(\nabla_x J(\theta, x^{(t)}, y)) \right)$$

ahol:

- α a lépésméret (step size), jellemzően $\alpha < \epsilon$,
- T az iterációk száma,
- $\Pi_{\mathcal{B}_\epsilon(x)}$ a projekciós operátor, amely a módosított mintát visszavetíti az eredeti x körüli ϵ sugarú ℓ_∞ gömbbe:

$$\Pi_{\mathcal{B}_\epsilon(x)}(z) = \text{clip}(z, x - \epsilon, x + \epsilon)$$

1.2.2. Az algoritmus lépései

1. Kezdeti pont megválasztása: $x^{(0)}$ lehet maga az eredeti x , vagy egy véletlen pont az $\mathcal{B}_\epsilon(x)$ környezetből (random restart).
2. Iteráció ($t = 0, \dots, T - 1$):
 - (a) Előreterjesztés és veszteségszámítás az aktuális $x^{(t)}$ pontra.
 - (b) Gradiens számítása: $g^{(t)} = \nabla_x J(\theta, x^{(t)}, y)$.
 - (c) Lépés a gradiens irányába: $\hat{x}^{(t+1)} = x^{(t)} + \alpha \cdot \text{sign}(g^{(t)})$.
 - (d) Projekció: $x^{(t+1)} = \Pi_{\mathcal{B}_\epsilon(x)}(\hat{x}^{(t+1)})$.
 - (e) Pixelértékek vágása a $[0, 1]$ tartományba.
3. A végső támadott kép: $x_{\text{adv}} = x^{(T)}$.

1.2.3. Célzott változat

A targeted PGD a célzott FGSM-hez hasonlóan a célosztály veszteségét minimalizálja:

$$x^{(t+1)} = \Pi_{\mathcal{B}_\epsilon(x)} \left(x^{(t)} - \alpha \cdot \text{sign}(\nabla_x J(\theta, x^{(t)}, y_{\text{target}})) \right)$$

1.2.4. Hiperparaméterek hatása

A PGD viselkedését három fő hiperparaméter határozza meg:

- ϵ : a megengedett maximális perturbáció. Nagyobb ϵ erősebb támadást tesz lehetővé, de a módosítás emberi szemmel is láthatóvá válhat.
- α : a lépésmagyság. Túl kis α lassú konvergenciát eredményez, túl nagy α pedig instabilitáshoz vezethet. Tipikus választás: $\alpha = \epsilon/T$ vagy $\alpha = 2.5 \cdot \epsilon/T$.
- T : az iterációk száma. Több iteráció általában erősebb támadást eredményez, de növeli a számítási költséget.

1.2.5. Előnyök és korlátok

A PGD az FGSM-nél lényegesen erősebb támadásokat képes generálni, mivel több kis lépésben közelíti a lokális optimumot. A random restart technikával kombinálva különösen hatékony. Hátránya, hogy T -szer annyi gradiens-számítást igényel, mint az FGSM, így nagyobb a számítási költsége.

Hivatkozások

- [1] Ian J. Goodfellow, Jonathon Shlens, Christian Szegedy. *Explaining and Harnessing Adversarial Examples*. arXiv:1412.6572, 2015.
- [2] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, Adrian Vladu. *Towards Deep Learning Models Resistant to Adversarial Attacks*. arXiv:1706.06083, 2018.